



Guía de operación, respaldo y recuperación

Sistema: Tiendita Maday

Cliente: La Familia

Versión: 2.0

Fecha de revisión: 16 de julio de 2026

Revisión examinada: 35a84c2 en main

Repositorio: https://github.com/CHidalgo405/web_integral.git

Audiencia: propietario del servicio y responsable técnico

Este documento es un procedimiento operativo verificable. Los objetivos de recuperación, horarios y responsables solo se convierten en compromiso cuando el cliente los aprueba y los registra en el acta de entrega o en un contrato de soporte.

1. Propósito, alcance y estado

La guía explica cómo publicar, supervisar, respaldar y recuperar Tiendita Maday sin depender de conocimiento informal del equipo. Complementa al **Manual técnico de instalación y configuración**, que cubre la preparación del entorno de desarrollo.

1.1 Componentes bajo operación

Componente	Tecnología observada	Responsabilidad operativa
Frontend	Angular 21.2, PWA estática	Interfaz para clientes, caja, inventario y administración.
Backend	Node.js 20 en contenedor, Express 4	API, autenticación, permisos y reglas de negocio.
Base de datos	PostgreSQL	Persistencia, transacciones, auditoría y migraciones.
Servicios externos	Google, PayPal, Cloudinary y correo	Inicio de sesión, pagos, imágenes y notificaciones.

El repositorio contiene archivos para Railway y contenedores. Esto demuestra una **ruta de despliegue prevista**, no que exista un ambiente productivo activo. Antes de la entrega deben registrarse URLs, propietarios y modo de cada servicio.

1.2 Diagnóstico operativo actual

Hallazgo	Estado	Acción antes de producción
Frontend y backend tienen railway.json.	Disponible	Vincular servicios, dominios y variables.
El Dockerfile del backend ejecuta node src/index.js.	Condicionado	Ejecutar npm run migrate como paso previo o incorporarlo al arranque validado.
npm start sí ejecuta migraciones, pero Railway usa el Dockerfile.	Riesgo	No asumir que las migraciones ocurren automáticamente.
docker-compose.yml está dentro de FrontEnd y referencia ./FrontEnd, ./Backend y ./Database.	Riesgo	Corregir ubicación/rutas y probar desde un clon limpio antes de usarlo.
La imagen local declara postgres:latest.	Riesgo	Fijar una versión mayor compatible para evitar actualizaciones inesperadas.
No se ejecutó restauración en la revisión actual.	Pendiente alto	Realizar simulacro en una base aislada y adjuntar evidencia.

2. Responsabilidad y control de acceso

Actividad	Propietario del negocio	Responsable técnico	Evidencia mínima
Aprobar precios, roles y configuración	Aprueba	Implementa	Solicitud o acta.
Custodiar cuentas y facturación	Responsable	Acceso delegado	Inventario de cuentas.
Desplegar y migrar	Informado	Responsable	Registro de versión.
Crear respaldos	Verifica	Ejecuta	Bitácora y checksum.
Probar restauración	Aprueba resultado	Ejecuta	Acta del simulacro.
Atender alertas	Decide prioridad	Diagnostica	Ticket o bitácora.
Gestionar incidente de seguridad	Decide comunicación	Contiene y documenta	Informe de incidente.
Revocar accesos al cierre	Aprueba	Ejecuta	Lista de accesos cerrada.

Reglas mínimas:

- cada persona debe usar una cuenta propia;
- activar segundo factor donde el proveedor lo permita;
- limitar producción al personal indispensable;
- revisar accesos trimestralmente y al salir un integrante;
- no copiar secretos en Git, documentos, capturas, tickets públicos ni chats sin cifrar.

3. Inventario de configuración

Las variables se configuran en el servicio correspondiente. La bitácora registra **quién** configuró el valor y **cuándo**, pero nunca almacena el secreto.

Variable	Servicio	Sensible	Uso y comprobación
DATABASE_URL	Backend	Sí	Conexión PostgreSQL; probar conexión y migraciones.
JWT_SECRET	Backend	Sí	Firma de acceso; usar valor largo, aleatorio y distinto.
JWT_REFRESH_SECRET	Backend	Sí	Renovación; no reutilizar JWT_SECRET.
ACCESS_TOKEN_EXPIRATION	Backend	No	Vigencia de acceso; validar política aprobada.
FRONTEND_URL	Backend	No	Origen permitido y enlaces; usar el dominio exacto.
BACKEND_URL	Backend	No	Dirección pública reportada por la API/OpenAPI.
PAYPAL_CLIENT_ID / PAYPAL_SECRET	Backend	Sí	Cobro; confirmar sandbox o live antes de cada prueba.
PAYPAL_API_BASE / PAYPAL_CURRENCY	Backend	No	Entorno PayPal y moneda MXN.
GOOGLE_CLIENT_ID	Ambos	Parcial	Inicio con Google; registrar orígenes autorizados.
CLOUDINARY_URL	Backend	Sí	Almacenamiento de imágenes.
EMAILJS_*	Backend	Sí	Verificación y recuperación de contraseña.
API_BASE_URL	Build frontend	No	URL del backend terminada en /api.
PORT	Backend	No	Puerto asignado por la plataforma.
POS_TEST_DATABASE_URL	Pruebas	Sí	Base desechable para integración POS.
INVENTORY_TEST_DATABASE_URL	Pruebas	Sí	Base desechable para integración de inventario.

El código admite valores de desarrollo para JWT. Es obligatorio definir secretos propios antes de publicar. La ausencia de una variable externa debe provocar una decisión explícita: configurar, desactivar el flujo o posponer la salida.

3.1 Ficha de ambiente

Dato	Desarrollo	Pruebas/ensayo	Producción
URL frontend	_____	_____	_____
URL backend	_____	_____	_____
PostgreSQL/proyecto	_____	_____	_____
Commit desplegado	_____	_____	_____
PayPal	No aplica/sandbox	Sandbox	_____
Responsable	_____	_____	_____
Último respaldo válido	_____	_____	_____

4. Liberación inicial y actualizaciones

4.1 Puerta de salida

No desplegar a producción si falta alguno de estos controles:

- ☐ Revisión exacta aprobada y rama protegida.
- ☐ Compilación frontend con código 0.
- ☐ Pruebas backend sin fallas; integraciones críticas ejecutadas sobre bases desechables.
- ☐ Suite frontend ejecutable y aprobada.
- ☐ Respaldo previo válido si ya existen datos.
- ☐ Migraciones ensayadas y responsable asignado.
- ☐ Variables y dominios verificados sin exponer secretos.
- ☐ PayPal en el modo autorizado.
- ☐ Plan de reversión y versión anterior disponibles.
- ☐ Ventana de mantenimiento comunicada.

La revisión 35a84c2 no cumple todavía todos estos puntos: el reporte de pruebas registra suite frontend no ejecutable, dos integraciones backend omitidas y restauración pendiente.

4.2 Secuencia de despliegue en Railway

1. Sincronice `main` y registre el hash que se liberará.
2. Cree o seleccione PostgreSQL; no reutilice una base de pruebas.
3. Para una instalación nueva, aplique `Arquitectura/Database/init.sql` en un entorno controlado.
4. Antes de iniciar el contenedor del backend, ejecute `npm run migrate` desde `Arquitectura/Backend` y confirme su salida.
5. Despliegue el backend con Root Directory `mi_cliente (la familia)/Arquitectura/Backend`.
6. Compruebe `/y/api-docs`; valide autenticación y conexión a datos.
7. Despliegue el frontend con Root Directory `mi_cliente (la familia)/Arquitectura/FrontEnd` y `API_BASE_URL` correcto.
8. Regrese al backend y limite `FRONTEND_URL` al dominio definitivo.
9. Ejecute pruebas de humo y registre el resultado.

No interrumpa una migración ni marque manualmente un archivo como aplicado. Si falla, conserve el error completo, detenga la liberación y evalúe la base antes de reintentar.

4.3 Pruebas de humo posteriores

- [] API y documentación responden por HTTPS.
- [] Frontend abre en una ventana privada sin errores visibles.
- [] Administrador y un rol operativo pueden iniciar sesión.
- [] Catálogo y detalle de producto cargan.
- [] Un origen no autorizado es rechazado por CORS.
- [] Correo de verificación/recuperación llega al destino de prueba.
- [] Cloudinary guarda y entrega una imagen de prueba.
- [] PayPal muestra el modo esperado; no hacer cobro live sin autorización.
- [] Venta POS de prueba y ajuste de inventario conservan trazabilidad.

5. Operación rutinaria

5.1 Calendario mínimo

Frecuencia	Actividad	Evidencia
Diaria	Revisar disponibilidad, errores críticos y respaldo lógico.	Monitor/bitácora.
Semanal	Revisar fallos HTTP, correo, imágenes, pagos y capacidad.	Resumen semanal.
Mensual	Actualizar dependencias primero en ensayo; revisar costos y accesos.	Ticket de mantenimiento.
Trimestral	Probar restauración, revisar privilegios y proveedores.	Acta del simulacro.
Antes de cada liberación	Respaldo, pruebas, migraciones y rollback.	Registro de versión.
Al cambiar personal	Revocar accesos y rotar secretos compartidos.	Lista de control.

5.2 Señales a vigilar

- disponibilidad y tiempo de respuesta del frontend y la API;
- errores HTTP 5xx, bloqueos de autenticación y fallos de CORS;
- conexiones, espacio y latencia de PostgreSQL;
- migraciones fallidas o pendientes;
- fallas de correo, carga de imágenes y respuesta de PayPal;
- diferencias entre venta, inventario y movimiento de caja;
- incremento anormal de errores o intentos de acceso.

Nunca guarde contraseñas, tokens, datos de tarjeta ni información completa de clientes en logs o tickets. Las capturas deben ocultar datos personales y credenciales.

5.3 Registro de liberación

Campo	Valor
Fecha y ventana	_____
Commit / versión	_____
Responsable	_____
Respaldo previo / checksum	_____
Migraciones aplicadas	_____
Resultado de pruebas	_____
Incidencias / rollback	_____
Aprobación del propietario	_____

6. Política de respaldo

Los siguientes valores son una propuesta inicial que el cliente debe aprobar.

Elemento	Recomendación
RPO: pérdida máxima admisible	24 horas; reducir según volumen y criticidad de ventas.
RTO: tiempo objetivo	4 horas dentro del horario de soporte acordado.
Respaldo lógico PostgreSQL	Diario y antes de cambios de alto riesgo.
Retención	7 diarios, 4 semanales y 6 mensuales.
Copias	Proveedor más una copia cifrada fuera del proveedor.
Integridad	SHA-256 registrado después de cada exportación.
Prueba de restauración	Trimestral y antes de cambios de alto riesgo.
Responsable	Persona nombrada en el acta.

Los respaldos administrados de Railway no sustituyen una copia lógica exportable bajo control del cliente. Las imágenes de Cloudinary y la configuración de terceros también requieren inventario o plan de recuperación; pg_dump solo protege PostgreSQL.

6.1 Crear y custodiar una copia lógica

Desde un equipo seguro con cliente PostgreSQL compatible:

```
pg_dump --format=custom --no-owner --no-acl --file=tiendita_YYYYMMDD_HHMM.dump  
"<DATABASE_URL>"
```

Después:

1. confirme que el proceso terminó con código 0 y que el archivo no está vacío;
2. calcule una suma SHA-256 y regístrela;
3. cifre la copia o almacénela en un repositorio cifrado con acceso limitado;
4. registre entorno, versión de PostgreSQL, tamaño, responsable y motivo;
5. verifique que la copia externa se puede leer;
6. aplique retención sin borrar la última copia mensual válida.

Antes de migraciones, cambios masivos de catálogo o correcciones directas de datos, genere un respaldo adicional etiquetado con el commit y el motivo.

7. Restauración controlada

Nunca practique sobre producción. La primera restauración se realiza en una base nueva, aislada y sin integraciones live.

1. Declare el incidente y detenga escrituras si hay riesgo de corrupción.
2. Defina el punto de recuperación y estime transacciones que quedarían fuera.
3. Seleccione la copia por fecha, checksum, entorno y compatibilidad.
4. Cree una base PostgreSQL vacía y desechable.
5. Restaure con `pg_restore --clean --if-exists --no-owner --no-acl --dbname="<TEST_DATABASE_URL>" archivo.dump`.
6. Ejecute las migraciones de la versión de aplicación que se utilizará.
7. Compare conteos y relaciones: usuarios, productos, inventario, pedidos, caja y movimientos.
8. Ejecute una venta de prueba y un ajuste de inventario sin proveedores live.
9. Documente tiempos reales, diferencias y errores.
10. Obtenga autorización del propietario antes de sustituir producción.
11. Cambie la conexión, publique y repita las pruebas de humo.
12. Preserve evidencia y redacte el informe posterior al incidente.

7.1 Criterios de aprobación

- la base abre sin errores de esquema;
- usuarios y roles esperados existen y respetan permisos;
- conteos y totales son plausibles frente a la fuente;
- venta, caja e inventario mantienen consistencia;
- no se usa PayPal live ni se envían correos reales durante el ensayo;

- se conocen fecha restaurada, pérdida de datos y tiempo real empleado;
- el propietario firma o acepta el resultado.

7.2 Registro del simulacro

Campo	Resultado
Fecha / responsable	_____
Copia y checksum	_____
Base aislada	_____
Inicio / fin / duración	_____
RPO y RTO obtenidos	_____
Conteos comparados	_____
Pruebas funcionales	_____
Errores y correcciones	_____
Aprobación	_____

8. Incidentes y reversión

Severidad	Ejemplo	Acción inicial
Crítica	Sitio caído, cobro inconsistente o pérdida de datos.	Contener, avisar al propietario y preservar evidencia inmediatamente.
Alta	Caja o checkout bloqueado para varios usuarios.	Habilitar alternativa operativa, diagnosticar y fijar actualización.
Media	Módulo secundario degradado.	Registrar, reproducir y programar corrección.
Baja	Texto, consulta o mejora cosmética.	Agregar al backlog con prioridad acordada.

Cada incidente debe conservar: fecha, reportante, ambiente, versión, impacto, acciones, evidencia, decisión, responsable y cierre.

8.1 Reversión de aplicación

Si el esquema sigue siendo compatible, despliegue el commit anterior y verifique primero la API y después el frontend. Si una migración cambió estructura o datos de manera incompatible, no revierta a ciegas: detenga escrituras, restaure una copia en aislamiento, determine impacto y obtenga autorización.

9. Transferencia al cliente

- ☐ El cliente administra GitHub, Railway, dominio y proveedores.
- ☐ Se entregaron URLs, cuentas propietarias y contactos de facturación.
- ☐ Se rotaron secretos compartidos durante el desarrollo.
- ☐ Existe una copia reciente con checksum y acceso comprobado.
- ☐ Se ejecutó y documentó una restauración.
- ☐ Se aprobaron RPO, RTO, retención y horario de soporte.
- ☐ Se documentaron costos, límites y renovaciones.
- ☐ Los canales de soporte publicados son reales.
- ☐ Se retiró acceso de quien ya no requiere administración.

10. Bitácora de respaldos

Fecha/hora	Entorno	Archivo/ID	Tamaño	SHA-256	Responsable	Restauración probada
_____	_____	_____	_____	_____	_____	Sí / No
_____	_____	_____	_____	_____	_____	Sí / No
_____	_____	_____	_____	_____	_____	Sí / No
_____	_____	_____	_____	_____	_____	Sí / No

11. Criterio de aceptación operativa

La guía puede considerarse implementada cuando existan, como mínimo:

1. inventario de ambientes, dominios y cuentas propietarias;
2. secretos de producción rotados y almacenados en el proveedor;
3. despliegue reproducible desde un commit identificado;
4. migraciones ejecutadas mediante un paso controlado;
5. pruebas automáticas y de humo aprobadas;
6. respaldo lógico con checksum en dos ubicaciones;
7. restauración documentada dentro de los objetivos acordados;
8. responsables, soporte y renovaciones aceptados por el cliente.

Estado en la revisión 35a84c2: condicionado. Existe una base documental y técnica para operar, pero la restauración, las pruebas integrales omitidas, la suite frontend y las inconsistencias de contenedores deben resolverse o aceptarse expresamente antes de declarar preparación productiva.